Data Management VO

07 APIs (JDBC, DB-API, ORM)

Shafaq Siddiqi - Lucas Iacono

Know-Center & Graz University of Technology, Austria

Computer Science and Biomedical Engineering
Institute of Human-Centred Computing

Course calendar

- Part 2: Physical design, query processing, APIs, and NoSQL
  - **(6) Mon. 11.05.2026: Database APIs + Exercise 3**

  - No lecture on 18.05.2026 (lecture free time to focus on Ex 3)
  - No lecture on 25.05.2026 (Whit Monday)

  - (8) Mon. 01.06.2026: NoSQL + Data Spaces

  - Mon. 08.06.2026: No lecture (time for VO exam #3)

  - **VO exam #3** on 15.06.2025 (means no lecture)

  Summer break

Last week

- **Physical Design and Transaction Processing**
  - Index structures
  - B-trees

- Questions or comments?
  - Please also see Discord

This week: Database APIs

- Call-level Interfaces (ODBC/JDBC/DB-API) and Embedded SQL
- Object-Relational Mapping (ORM) Frameworks

This week: Database APIs

- **Call-level Interfaces (ODBC/JDBC/DB-API) and Embedded SQL**
- **Object-Relational Mapping (ORM) Frameworks**

- **Experiences**
  - Database access via Java?
  - Database access via Python?
  - ORM?

What's an API and why should we care?

- Application Programming Interface (API)
  - Defined **set of functions or protocols**
    for system or component communication

What's an API and why should we care?

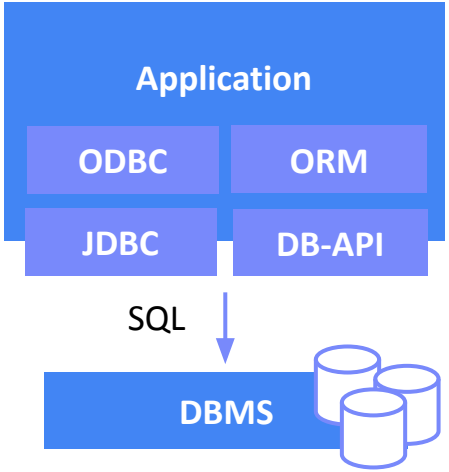
- Application Programming Interface (API)
  - Defined **set of functions or protocols** for system or component communication
  - Interface independent of concrete implementation **decoupling of applications** from underlying libraries / systems
  - API stability of utmost importance

What's an API and why should we care?

- Examples
  - **Linux:** kernel-user space API □ system calls, POSIX (Portable Operating System Interface)
  - **Cloud Services:** often dedicated REST (Representational State Transfer) APIs

What's an API and why should we care?

- Examples
  - **Linux:** kernel-user space API system calls, POSIX (Portable Operating System Interface)
  - **Cloud Services:** often dedicated REST (Representational State Transfer) APIs
  - **DB Access: ODBC (Open Database Connectivity)/ JDBC/DB-API and ORM frameworks**



What's an API and why should we care?

- Why should we care about database APIs?
  - E.g., data-driven research
  - Query data from database and analyze in data science application

Call-level Interfaces (ODBC/JDBC/DB-API) and Embedded SQL

TU Graz
HCC Institute of HUMAN-CENTRED COMPUTING

Call-level Interfaces vs Embedded SQL

- #1 Call-level Interfaces
  - **API of defined functions for dynamic SQL** (changes possible at runtime)
  - Syntax / type checks also only possible at runtime
  - **Examples:** ODBC (C/C++), JDBC (Java), DB-API (Python)

Call-level Interfaces vs Embedded SQL

- #1 Call-level Interfaces
  - **API of defined functions for dynamic SQL** (changes possible at runtime)
  - Syntax / type checks also only possible at runtime
  - **Examples:** ODBC (C/C++), JDBC (Java), DB-API (Python)

- #2 Embedded SQL
  - **Embedded SQL in host language** (typically static)
  - **Precompiler** to translate SQL into host language
    - SQL syntax and type checking at compile time, but SQL is fixed in code
  - **Examples:** ESQL (C/C++), SQLJ (Java), in-line SQL (Python)

Call-level Interfaces vs Embedded SQL

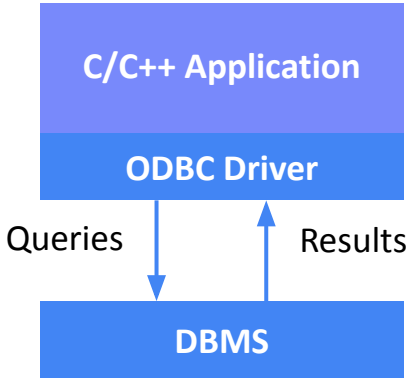
- #1 Call-level Interfaces
  - **API of defined functions for dynamic SQL** (changes possible at runtime)
  - Syntax / type checks also only possible at runtime
  - **Examples:** ODBC (C/C++), JDBC (Java), DB-API (Python)
    executeQuery(sqlString)

- #2 Embedded SQL
  - **Embedded SQL in host language** (typically static)
  - **Precompiler** to translate SQL into host language
    □ SQL syntax and type checking at compile time, but SQL is fixed in code
  - **Examples:** ESQL (C/C++), SQLJ (Java), in-line SQL (Python)

In practice, very limited relevance □ focus on CLI!

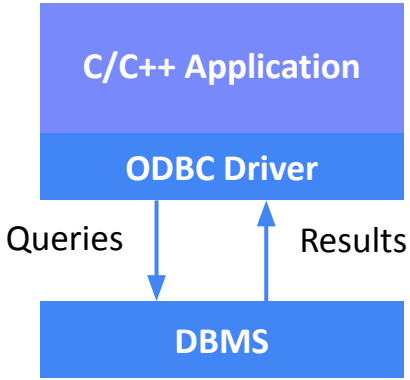
CLI: ODBC and JDBC Overview

- Open Database Connectivity (ODBC)
  - **API for accessing databases independent of DBMS from C/C++**
  - Developed and released in **1992**
  - **All relational DBMS have ODBC implementations**



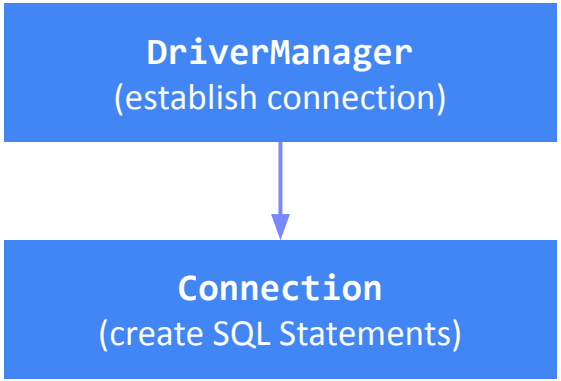
CLI: ODBC and JDBC Overview

- Open Database Connectivity (ODBC)
  - **API for accessing databases independent of DBMS from C/C++**
  - Developed and released in **1992**
  - **All relational DBMS have ODBC implementations**

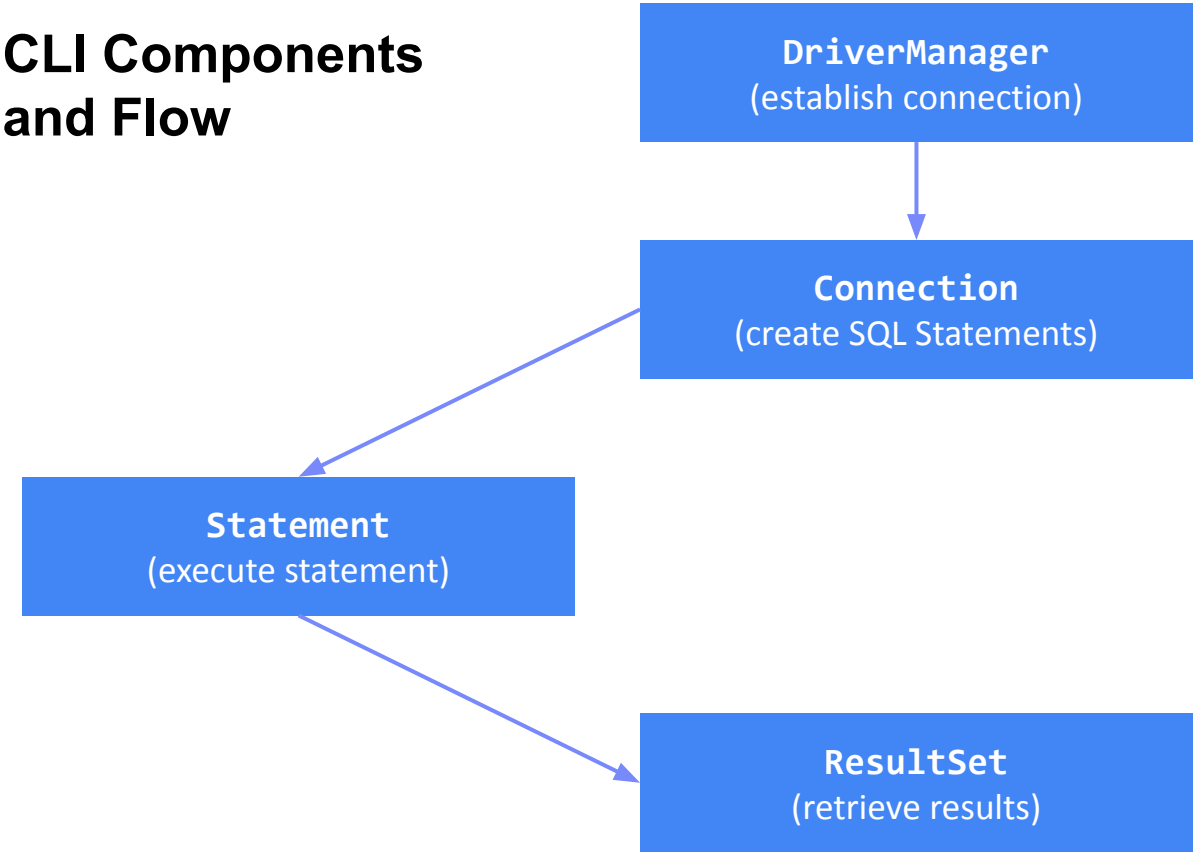


- Java Database Connectivity (JDBC)
  - **API for accessing databases independent of DBMS from Java**
  - Developed and released by Sun in **1997**
  - Most relational DBMS have JDBC implementations
  - Different types of drivers (e.g., pure Java implementations or ODBC wrappers)

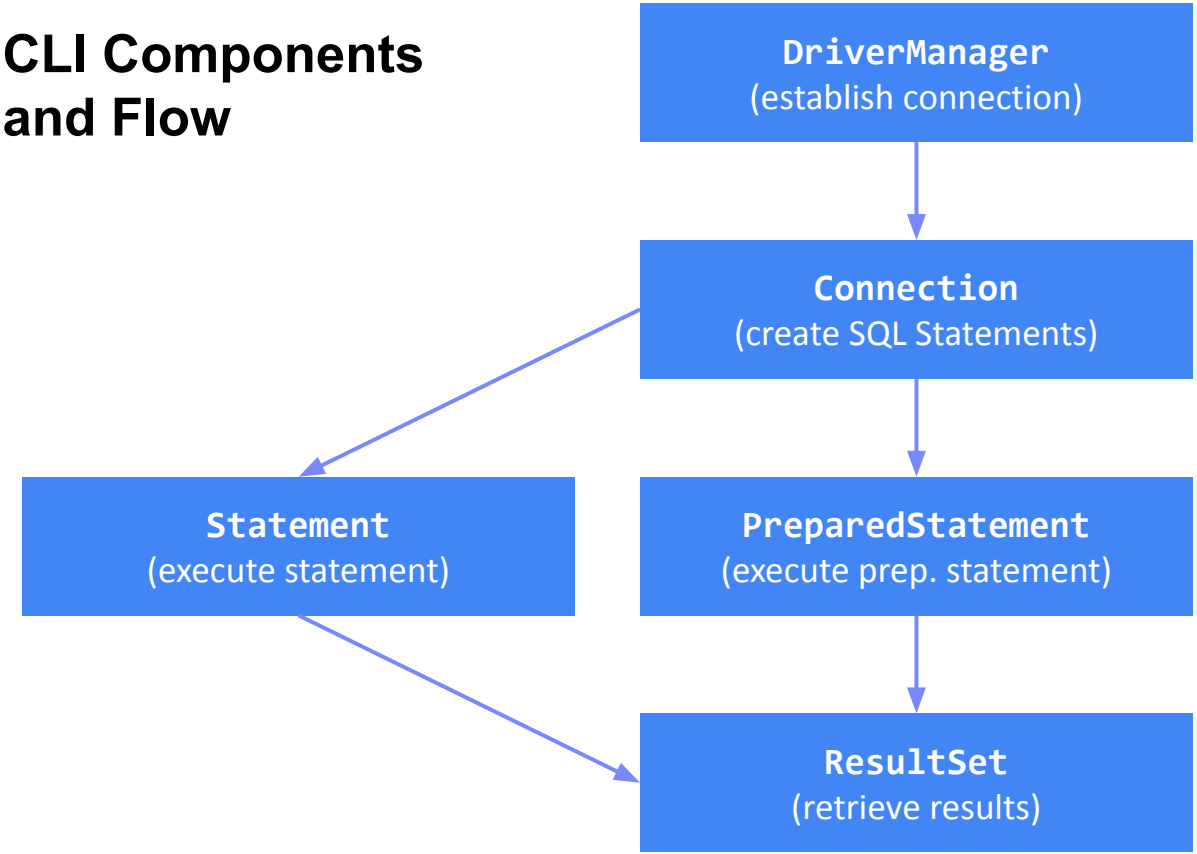
CLI Components and Flow



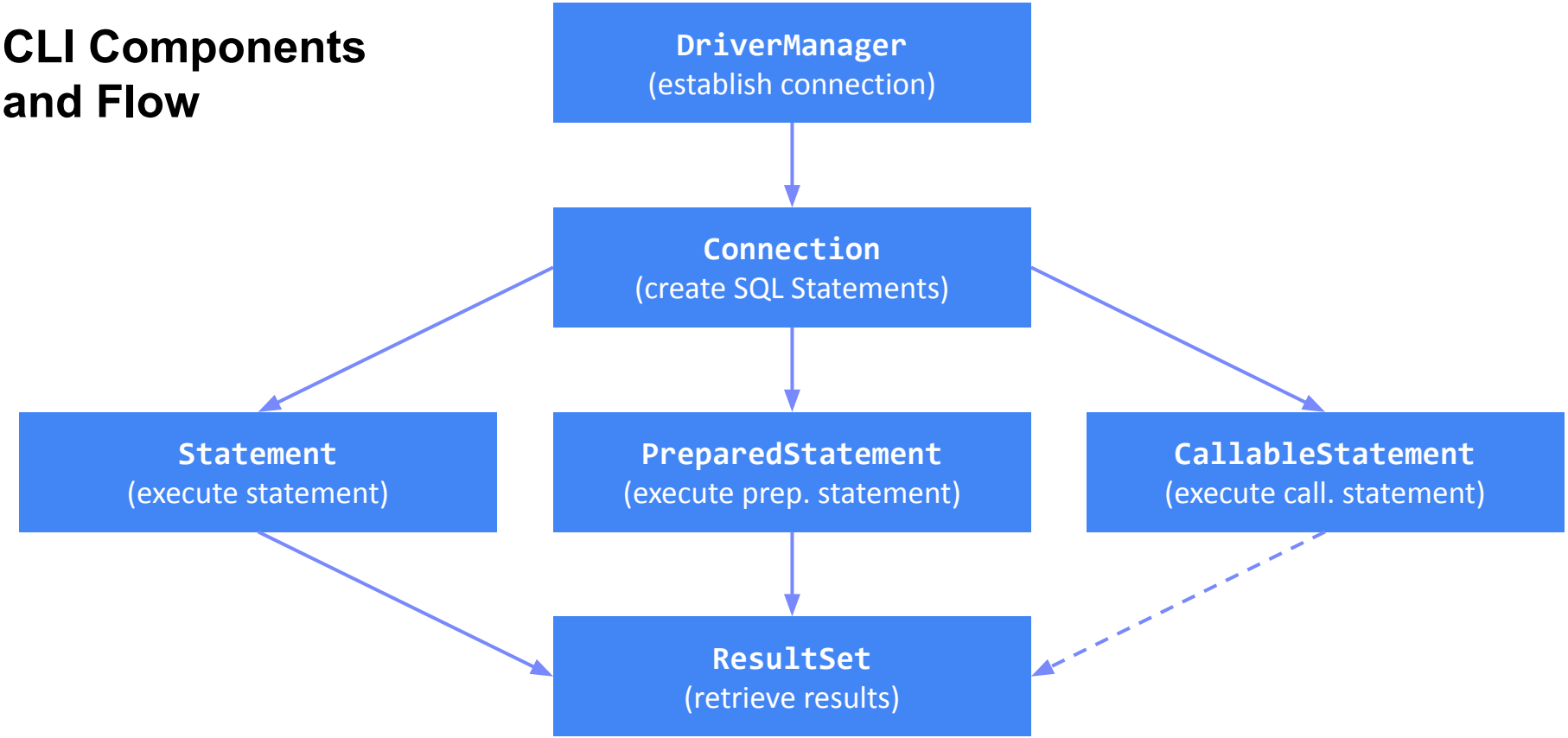
CLI Components and Flow



CLI Components and Flow



CLI Components and Flow



#1 JDBC Connection Handling

- Establishing a Connection
  - **DBMS-specific URL strings** including db, host, port, and database name + credentials
  - Objects representing user-specific DB sessions

```
Connection conn = DriverManager
   .getConnection("jdbc:postgresql:"+
   "//localhost:5432/db1234567",
   username, password);

…

conn.close()
```

#2 JDBC Statements

- Execute `Statement`
  - Use for simple SQL statements w/o parameters
  - ExecuteQuery vs. ExecuteUpdate

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql1);
...
stmt.close();
```

#2 JDBC Statements

- Execute `Statement`
  - Use for simple SQL statements w/o parameters
  - ExecuteQuery vs. ExecuteUpdate

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql1);
…
stmt.close();

Statement stmt = conn.createStatement();
int rows = stmt.executeUpdate(sql2);
…
stmt.close();
```

#2 JDBC Statements

- Execute `Statement`
  - Use for simple SQL statements w/o parameters
  - ExecuteQuery vs. ExecuteUpdate

Beware of SQL injection

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql1);
…
stmt.close();

Statement stmt = conn.createStatement();
int rows = stmt.executeUpdate(sql2);
…
stmt.close();
```

Beware of SQL Injection

- Problematic SQL String Concatenation

```
INSERT INTO Students (Lname, Fname)
   VALUES (‘“+ @lname +”‘,’“+ @fname +”’);”;
```

Beware of SQL Injection

- Problematic SQL String Concatenation

```
INSERT INTO Students (Lname, Fname)
   VALUES (‘“+ @lname +”‘,’“+ @fname +”’);”;
```

What could go wrong here?

Beware of SQL Injection

- Problematic SQL String Concatenation

```
INSERT INTO Students (Lname, Fname)
   VALUES (‘“+ @lname +”‘,’“+ @fname +”’);”;
```

What could go wrong here?

- Possible SQL-Injection Attack

```
INSERT INTO Students (Lname, Fname) VALUES (‘Smith‘,’Robert’);
DROP TABLE Students; --’);
```

#2 JDBC Statements

- Process ResultSet
  - Iterator-like cursor w/ on-demand fetching
  - Start of cursor before the first result in the ResultSet
  - Attribute access via column names

```
ResultSet rs = stmt.executeQuery(
   “SELECT SID, LName FROM Students”);
```

#2 JDBC Statements

- Process ResultSet
  - Iterator-like cursor w/ on-demand fetching
  - Start of cursor before the first result in the ResultSet
  - Attribute access via column names

```
ResultSet rs = stmt.executeQuery(
   “SELECT SID, LName FROM Students”);

List<Student> ret = new ArrayList<>();
while( rs.next() ) {
   int id = rs.getInt(“SID”);
   String name = rs.getString(“LName”);
   ret.add(new Student(id, name));
}
```

#3 JDBC Prepared Statements

- Execute `PreparedStatement`
  - Use for precompiling SQL statements w/ input params
  - Parameter index starts with 1
  - **Precompile SQL once**, and execute many times

```
PreparedStatement pstmt =
    conn.prepareStatement(
      “INSERT INTO Students
VALUES(?,?)”);

for( Student s : students ) {
    pstmt.setInt(1, s.getID());
    pstmt.setString(2, s.getName());
    pstmt.executeUpdate();
}

pstmt.close();
```

#3 JDBC Prepared Statements

- Execute `PreparedStatement`
  - Use for precompiling SQL statements w/ input params
  - Parameter index starts with 1
  - **Precompile SQL once**, and execute many times

- **Performance**
- **No danger of SQL injection**

```
PreparedStatement pstmt =
    conn.prepareStatement(
      "INSERT INTO Students
VALUES(?,?)");

for( Student s : students ) {
    pstmt.setInt(1, s.getID());
    pstmt.setString(2, s.getName());
    pstmt.executeUpdate();
}

pstmt.close();
```

#4 JDBC Callable Statements

- Execute `CallableStatement`
  - Used for call of a procedure **stored procedure (SQL lecture)**
  - Explicit registration of output parameters (similar to C++)

#4 JDBC Callable Statements

- Execute `CallableStatement`
  - Used for call of a procedure → **stored procedure (SQL lecture)**
  - Explicit registration of output parameters (similar to C++)
  - Example

```
CallableStatement cstmt = conn.prepareCall(
   "{CALL prepStudents(?, ?)}");

cstmt.setInt(1, 2024);
cstmt.registerOutParameter(2, Types.INTEGER);
cstmt.executeQuery();

int rows = cstmt.getInt(2);
```

Psycopg (Python PostgreSQL Adapter)

- Overview Psycopg
  - Implements **Python Database API Specification**
  - Install via `pip install psycopg2` (or similar via conda)
    - Pip comes with standard Python installation: https://www.python.org/downloads/
    - Conda via Anaconda installation: https://www.anaconda.com/

Psycopg (Python PostgreSQL Adapter)

- Overview Psycopg
  - Implements **Python Database API Specification**
  - Install via `pip install psycopg2` (or similar via conda)
    - Pip comes with standard Python installation: https://www.python.org/downloads/
    - Conda via Anaconda installation: https://www.anaconda.com/

- Establish Connection

```
import psycopg2

conn = psycopg2.connect(
    host="localhost", port="5432",
    database="db1234567", user=username,
    password=password)
```

Psycopg (Python PostgreSQL Adapter)

- Execute Statements
  - Use local cursors
  - Commit to persist

```
cur = conn.cursor()
cur.execute("INSERT INTO Students VALUES(...)")
conn.commit()
```

Psycopg (Python PostgreSQL Adapter)

- Execute Statements
  - Use local cursors
  - Commit to persist

```
cur = conn.cursor()
cur.execute("INSERT INTO Students VALUES(...)")
conn.commit()
```

- Process Result Sets
  - fetchall vs fetchone
  - Access properties via indices

```
cur.execute("SELECT SID, LName FROM Students")
students = cur.fetchall()
for row in students:
    print("SID = ", row[0], end = ',')
    print("Lname = ", row[1])
```

Psycopg (Python PostgreSQL Adapter)

- Execute Prepared Statements
  - Performance
  - SQL injection

```
cur = conn.cursor()
sql = “INSERT INTO Students VALUES(%s, %s)”
for s in students:
    cur.execute(sql, (s.getID(),s.getName()))
conn.commit()
```

Psycopg (Python PostgreSQL Adapter)

- Execute Prepared Statements
  - Performance
  - SQL injection
- Execute Callable Statement
  - Result set
  - No output parameters
  - difference to ODBC/JDBC

```
cur = conn.cursor()
sql = "INSERT INTO Students VALUES(%s, %s)"
for s in students:
    cur.execute(sql, (s.getID(),s.getName()))
conn.commit()

cur = conn.cursor()
cur.callproc("prepStudents", (2024, out))
rows = cur.fetchone()
```

Psycopg (Python PostgreSQL Adapter)

- Execute Prepared Statements
  - Performance
  - SQL injection

```
cur = conn.cursor()
sql = “INSERT INTO Students VALUES(%s, %s)”
for s in students:
    cur.execute(sql, (s.getID(),s.getName()))
conn.commit()
```

- Execute Callable Statement
  - Result set
  - No output parameters
  - difference to ODBC/JDBC

```
cur = conn.cursor()
cur.callproc(“prepStudents”, (2024, out))
rows = cur.fetchone()
```

- Close Connection

```
cur.close()
conn.close()
```

Psycopg (Work with csv files)

- csv.reader
  - Standard functionality in Python
  - In general, an alternative is csv.dictReader

Psycopg (Work with csv files)

- csv.reader
  - Standard functionality in Python
  - In general, an alternative is csv.dictReader
  - Example:

```
reader = csv.reader(open(file))
for row in reader:
    for element in row:
    [do something]
```

Psycopg (Work with csv files)

- csv.reader
  - Standard functionality in Python
  - In general, an alternative is csv.dictReader
  - Example:

```
reader = csv.reader(open(file))
for row in reader:
    for element in row:
    [do something]
```

  - More on how to deal with databases within Python in **“Databases”** course in WS
- Demo https://colab.research.google.com/drive/1PacKhZ6X_YWN6avUx8qTCxgARD10CpwZ?usp=sharing

Call-level Interfaces (ODBC/JDBC/DB-API) and Embedded SQL

Questions? Comments?

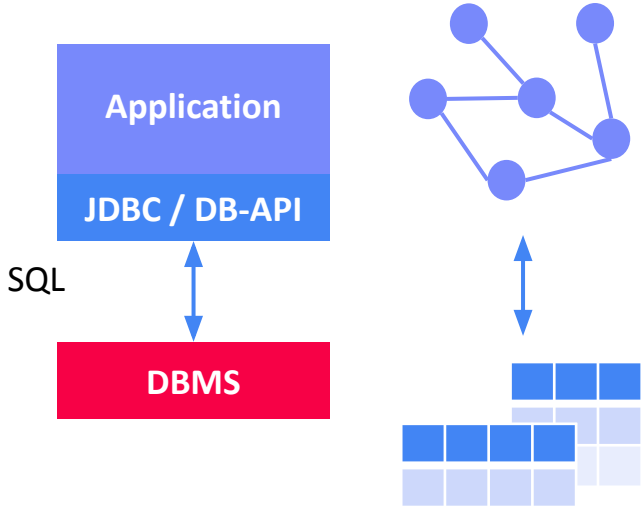
Exercise 3

link

Object-Relational Mapping Frameworks (ORM)

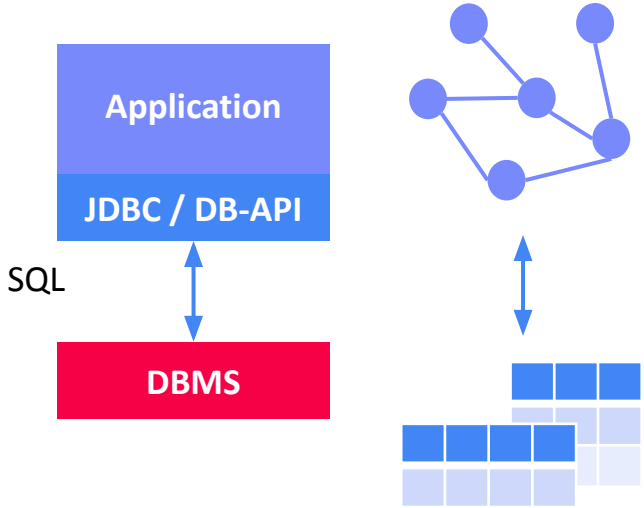
The “Impedance Mismatch” Argument

- **Problem Description**
  - Applications rely on **object-oriented programming languages** with hierarchies or graphs of objects
  - Data resides in **normalized “flat” tables**



The “Impedance Mismatch” Argument

- **Problem Description**
  - Applications rely on **object-oriented programming languages** with hierarchies or graphs of objects
  - Data resides in **normalized “flat” tables**

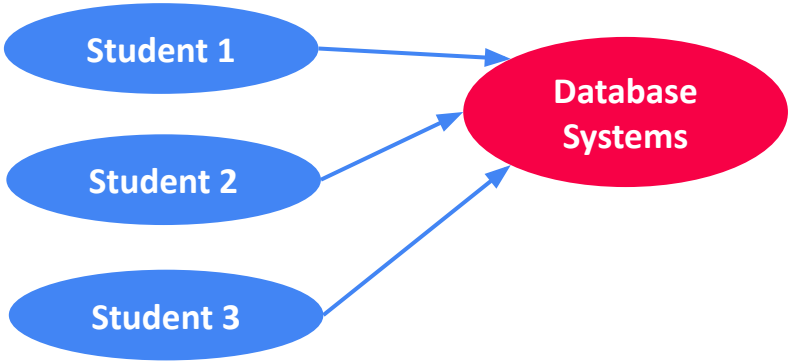


ORMs help

bridging this structural/behavioral gap

The “Impedance Mismatch” Argument

- **Example**
  - `SELECT * FROM Students`
  - ```
    SELECT C.Name, C.ECTS FROM
      Courses C, Attendance A
      WHERE C.CID = A.CID
        AND A.SID = 7;
    ```



→Interacting with a”student” object involves several database tables (e.g. Courses, Attendance) and complex queries
    ```

Overview Object-Relational Mapping

- **Goals of ORM Tools**
  - Automatic **handling of object persistence lifecycle** and querying of the underlying data stores (e.g., RDBMS)
  - Reduced development effort **developer productivity**

Overview Object-Relational Mapping

- **Common High-Level Architecture**
  - #1 Persistence definition (meta data □ e.g., **XML**)

Meta data

Class Student → **table** STUDENTS
attribute id → **primary key** ID
attribute name → **column** NAME

```
<hibernate-mapping>
    <class name="Student"
table="STUDENTS">
        <id name="id" column="ID">
            <generator class="native"/>
        </id>
        <property name="name"
column="NAME"/>
    </class>
</hibernate-mapping>
```

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import declarative_base

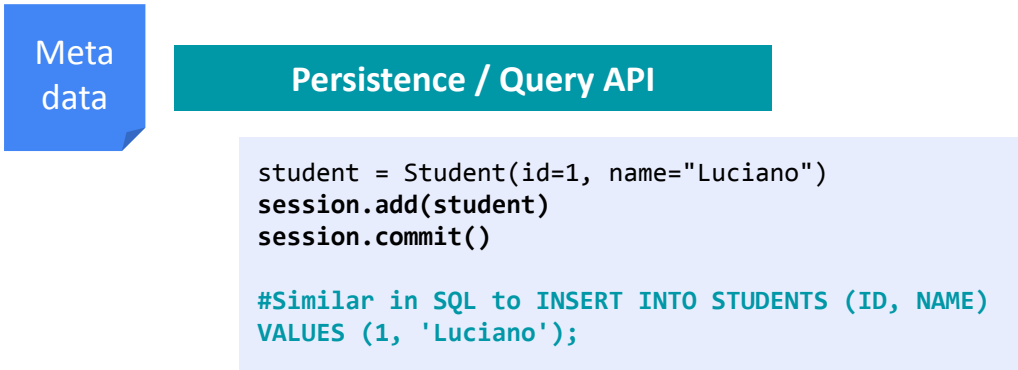
Base = declarative_base()
       class Student(Base):
       __tablename__ = "STUDENTS"
       id = Column("ID", Integer,
       primary_key=True)
       name = Column("NAME", String)
```

```
@Entity
public class Student {
   @Id
   private int SID = -1;
   private String Fname;
   private String Lname;
   @ManyToMany
   private List<Course> ...
```

HCC Institute of HUMAN-CENTRED COMPUTING

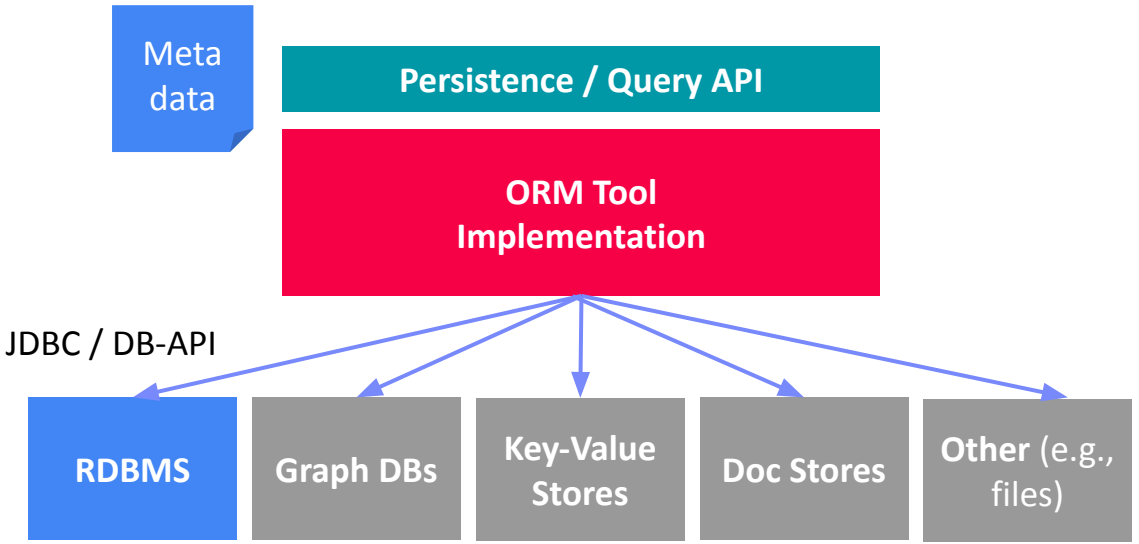
Overview Object-Relational Mapping

- **Common High-Level Architecture**
  - **#1** Persistence definition (meta data □ e.g., XML)
  - **#2** Persistence/Query API



Overview Object-Relational Mapping

- **Common High-Level Architecture**
  - **#1** Persistence definition (meta data □ e.g., XML)
  - **#2** Persistence/Query API
  - **#3** ORM implementation



History and Landscape

- **History of ORM Tools (aka persistence frameworks)**
  - Since 2000 Java **Entity Beans**
  - Since 2001 **Hibernate** framework (quite popular)
  - 2006 **JPA** (**Java Persistence API**), reference implementation **TopLink**
  - 2013 **JPA 2**, reference implementation **EclipseLink**
  - **2013 - today:** different ORM tools emerged (**e.g., SQLAlchemy**)

History and Landscape

- **Example frameworks/technologies**
  - http://java-source.net/open-source/persistence
  - ORMs for Python: SQLAlchemy, Tortoise, Django …





#1 JPA – Class Definition and Meta Data

- **Entity Classes**
  - **Define persistent classes** via annotations
  - Add details for IDs, relationship types (e.g., ManyToMany, etc.)
    - Each student can be enrolled to many courses and each course can have many students

```
@Entity
public class Student {
   @Id
   private int SID = -1;
   private String Fname;
   private String Lname;
   @ManyToMany
   private List<Course> ...
}
```

#1 JPA – Class Definition and Meta Data

- **Persistence Definition**
  - Separate XML meta data
    `META-INF/persistence.xml`
  - Alternative to entity annotations

```
<?xml version="1.0" encoding="UTF-8" ?>
<persistence
      xmlns="http://xmlns.jcp.org/xml/ns/persistence"
      xmlns:xsi=... xsi:schemaLocation=...>
   <persistence-unit name="UniversityDB">
      <class>org.tugraz.Student</class>
      <class>org.tugraz.Course</class>
      <exclude-unlisted-classes/>
      <properties> ... </properties>
   </persistence-unit>
</persistence>
```

#1 JPA – Object Modification

- **Persistence**
  - Persist objects in database
  - Update and delete objects

```
EntityManager em = factory
   .createEntityManager();

Student s = new
   Student(7,”Jane”,”Smith”);
s.addCourse(new Course(...));
s.addCourse(new Course(...));

em.persist(s);

em.close
```

#1 JPA – Query Languages

- **JPQL: Java Persistence** Query Language
  - SQL-like object-oriented query language
  - Iterate over objects

```
EntityManager em = factory
   .createEntityManager();
Query q = em.createQuery(
   "SELECT s FROM Student s
      WHERE s.age > :age");
q.setParameter(“age”, 25);

Iterator iter = q
   .getResultList().iterator();
while( iter.hasNext() )
   print((Student)iter.next());
```

#2 Python ORM Mapping (SQLAlchemy)

- **SQLAlchemy**
  - Implements Python DB-API and provides ORM
  - Install via `pip install SQLAlchemy` (or similar via conda)

#2 Python ORM Mapping (SQLAlchemy)

- **SQLAlchemy**
  - Implements Python DB-API and provides ORM
  - Install via `pip install SQLAlchemy` (or similar via conda)

- **DB connection**

+ base object for persistence

```
db = create_engine(db_string)
base = declarative_base()
```

#2 Python ORM Mapping (SQLAlchemy)

- **Create mapping**

```
class Student(base):
    __tablename__ = „Students“
    SID = Column(String, primary_key=True)
    Lname = Column(String)
    Fname = Column(String)

base.metadata.create_all(db)
```

#2 Python ORM Mapping (SQLAlchemy)

- **Create mapping**

```
class Student(base):
    __tablename__ = „Students“
    SID = Column(String, primary_key=True)
    Lname = Column(String)
    Fname = Column(String)

base.metadata.create_all(db)
```

- **Create session**

```
session = sessionmaker(db)()
```

#2 Python ORM Mapping (SQLAlchemy) - CRUD

- **Create**

```
roxana = Student(Fname="Roxana"
                 Lname="Martinez")
session.add(roxana)
session.commit()
```

- **Read**

(could also contain filters)

```
students = session.query(Student)
for s in students:
    print(s.Lname)
```

#2 Python ORM Mapping (SQLAlchemy) - CRUD

- **Update**

```
roxana.Lname = „NewName“
session.commit()
```

- **Delete**

```
session.delete(roxana)
session.commit()
```

A Critical View on ORM

- **Advantages**
  - **Simple CRUD operations** (create/read/update/delete and simple queries)
  - **Application-centric development**
- **Disadvantages**
  - **Indirections** and complexity (metadata, mapping)
  - **Performance problems** (mapping is a hard problem)
  - **Dependence** on evolving framework APIs

 Awareness of strength and weaknesses

Conclusions

- Summary
  - **Call-level Interfaces (JDBC/DB-API)** as fundamental access technology
  - **Exercise 3**
  - **Object-Relational Mapping (ORM)** frameworks (**pros and cons**)
- Next week (18.05.2026)
  - **No lecture (time for Ex3)**
- Week after (25.05.2026)
  - **Whit Monday (no lecture)**
- Next lecture (01.06.2026)
  - **NoSQl & Data Spaces**
- **Course evaluation started in TUG-online please take some minutes to fill it!**